Task

Perform a dimensionality reduction of pre-trained word embeddings using t-SNE, visualize the 2-D embeddings to identify semantically similar word clusters, and provide an analysis report interpreting the observed relationships.

Load Pre-trained Word Embeddings

Subtask:

Load a pre-trained word embedding model (e.g., Word2Vec, GloVe) using the `gensim` library. This will provide the high-dimensional vectors for words.

Reasoning: The first step is to import the necessary `gensim` module for loading pre-trained word embeddings.

```
import gensim.downloader as api

print("gensim.downloader imported as api.")
```

```
gensim.downloader imported as api.
```

Reasoning: The previous code failed because the `gensim` library was not installed. I will install `gensim` using pip.

```
pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 60.5 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

Reasoning: Now that `gensim` is installed, I will import `gensim.downloader` as `api`, load the 'word2vec-google-news-300' model, and verify its vocabulary size and a sample word vector.

```
import gensim.downloader as api

print("gensim.downloader imported as api.")

# Load a pre-trained word embedding model
# Using a smaller model for quicker download and demonstration, but 'word2vec-google-news-300' can be used.
# For example, 'glove-wiki-gigaword-50' or 'word2vec-google-news-300'
print("Loading word2vec-google-news-300 model...")
word_vectors = api.load('word2vec-google-news-300')
print("Model 'word2vec-google-news-300' loaded successfully.")

# Verify the model
print(f"Number of words in vocabulary: {len(word_vectors.index_to_key)}")

# Access a vector for a common word
word = 'computer'
if word in word_vectors:
    print(f"Vector for '{word}': {word_vectors[word][:10]}...") # Display first 10 dimensions
```

```
    else:
        print(f"Word '{word}' not found in vocabulary.")
```

```
gensim.downloader imported as api.
Loading word2vec-google-news-300 model...
[==================================================] 100.0% 1662.8/1662.8MB downloaded
Model 'word2vec-google-news-300' loaded successfully.
Number of words in vocabulary: 3000000
Vector for 'computer': [ 0.10742188 -0.20117188  0.12304688  0.21191406 -0.09130859  0.21679688
 -0.13183594  0.08300781  0.20214844  0.04785156]...
```

Task

Select 40-50 meaningful words for visualization, explaining the rationale for their selection to ensure semantic diversity or focus on specific themes.

Select Words for Visualization

Subtask:

Choose 40-50 meaningful words to visualize. Briefly explain the rationale behind the selection to ensure semantic diversity or focus on specific themes relevant to your analysis.

Reasoning: I will create a list of 40-50 diverse words for visualization, covering various semantic categories such as professions, emotions, animals, food, technology, and nature, to ensure a comprehensive representation of semantic relationships.

```
selected_words = [
    'doctor', 'nurse', 'engineer', 'teacher', 'artist', 'scientist', 'musician', 'lawyer', 'pilot', 'chef',
    'happy', 'sad', 'angry', 'surprised', 'calm', 'excited', 'fear', 'joy', 'love', 'hate',
    'dog', 'cat', 'bird', 'fish', 'lion', 'elephant', 'tiger', 'wolf', 'snake', 'monkey',
    'apple', 'banana', 'orange', 'grape', 'bread', 'cheese', 'pizza', 'pasta', 'coffee', 'water',
    'computer', 'phone', 'internet', 'software', 'data', 'algorithm', 'robot', 'car', 'plane', 'train'
]

print(f"Selected {len(selected_words)} words for visualization.")
```

```
Selected 50 words for visualization.
```

The selected words aim to achieve semantic diversity, covering several broad categories:

- **Professions**: `doctor`, `nurse`, `engineer`, `teacher`, `artist`, `scientist`, `musician`, `lawyer`, `pilot`, `chef`
- **Emotions**: `happy`, `sad`, `angry`, `surprised`, `calm`, `excited`, `fear`, `joy`, `love`, `hate`
- **Animals**: `dog`, `cat`, `bird`, `fish`, `lion`, `elephant`, `tiger`, `wolf`, `snake`, `monkey`
- **Food & Drink**: `apple`, `banana`, `orange`, `grape`, `bread`, `cheese`, `pizza`, `pasta`, `coffee`, `water`
- **Technology & Transportation**: `computer`, `phone`, `internet`, `software`, `data`, `algorithm`, `robot`, `car`, `plane`, `train`

This diverse selection will allow us to observe how the pre-trained word embeddings cluster words within these categories and how different categories relate to each other in the reduced-dimension space. It helps in assessing the embedding model's ability to capture nuanced semantic relationships across various domains.

Extract Embeddings

Subtask:

Extract the high-dimensional vector embeddings for the selected words from the loaded pre-trained model.

Reasoning: I will extract the high-dimensional vector embeddings for the selected words, ensuring only words present in the pre-trained model are included, and then convert them into a NumPy array for further processing.

```
import numpy as np

word_embeddings = []
valid_words = []

for word in selected_words:
    if word in word_vectors:
        word_embeddings.append(word_vectors[word])
        valid_words.append(word)
    else:
        print(f"Warning: Word '{word}' not found in the pre-trained model vocabulary. Skipping.")

embeddings_array = np.array(word_embeddings)

print(f"Number of words for which embeddings were extracted: {len(valid_words)}")
print(f"Shape of embeddings_array: {embeddings_array.shape}")
```

```
Number of words for which embeddings were extracted: 50
Shape of embeddings_array: (50, 300)
```

Apply t-SNE for Dimensionality Reduction

Subtask:

Use scikit-learn's t-SNE implementation to reduce the dimensionality of the word embeddings from their original high dimension to 2-D, preparing them for plotting.

Reasoning: To reduce the dimensionality of the word embeddings to 2-D for visualization, I will import the `TSNE` class, instantiate it with appropriate parameters for reproducibility and optimal performance, and then apply the transformation to the `embeddings_array`.

```
from sklearn.manifold import TSNE

# Instantiate t-SNE with desired parameters
# n_components=2 for 2D visualization
# random_state for reproducibility
# perplexity and n_iter are important for t-SNE quality
print("Initializing t-SNE model...")
tsne = TSNE(n_components=2, random_state=42, perplexity=30, n_iter=1000)

# Apply t-SNE to the embeddings_array
print("Applying t-SNE transformation...")
tsne_2d_embeddings = tsne.fit_transform(embeddings_array)

print("t-SNE dimensionality reduction complete.")
print(f"Shape of 2D embeddings: {tsne_2d_embeddings.shape}")
```

```
Initializing t-SNE model...
Applying t-SNE transformation...
/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning: 'n_iter' was renamed to
  warnings.warn(
t-SNE dimensionality reduction complete.
Shape of 2D embeddings: (50, 2)
```

Reasoning: The previous code generated a `FutureWarning` because the `n_iter` parameter in `TSNE` was renamed to `max_iter` in scikit-learn version 1.5. I will update the code to use `max_iter` for future compatibility, while keeping the

functionality the same.

```
from sklearn.manifold import TSNE

# Instantiate t-SNE with desired parameters
# n_components=2 for 2D visualization
# random_state for reproducibility
# perplexity and max_iter are important for t-SNE quality
print("Initializing t-SNE model...")
tsne = TSNE(n_components=2, random_state=42, perplexity=30, max_iter=1000)

# Apply t-SNE to the embeddings_array
print("Applying t-SNE transformation...")
tsne_2d_embeddings = tsne.fit_transform(embeddings_array)

print("t-SNE dimensionality reduction complete.")
print(f"Shape of 2D embeddings: {tsne_2d_embeddings.shape}")
```

```
Initializing t-SNE model...
Applying t-SNE transformation...
t-SNE dimensionality reduction complete.
Shape of 2D embeddings: (50, 2)
```

Visualize 2-D Embeddings

Subtask:

Generate a scatter plot of the 2-D word embeddings using `matplotlib`. Label each point with its corresponding word to clearly identify clusters and relationships. Ensure legends are provided if different categories of words are chosen.

Reasoning: I need to install the `adjustText` library to prevent labels from overlapping in the t-SNE plot, as it is not part of the standard Python or scikit-learn distributions.

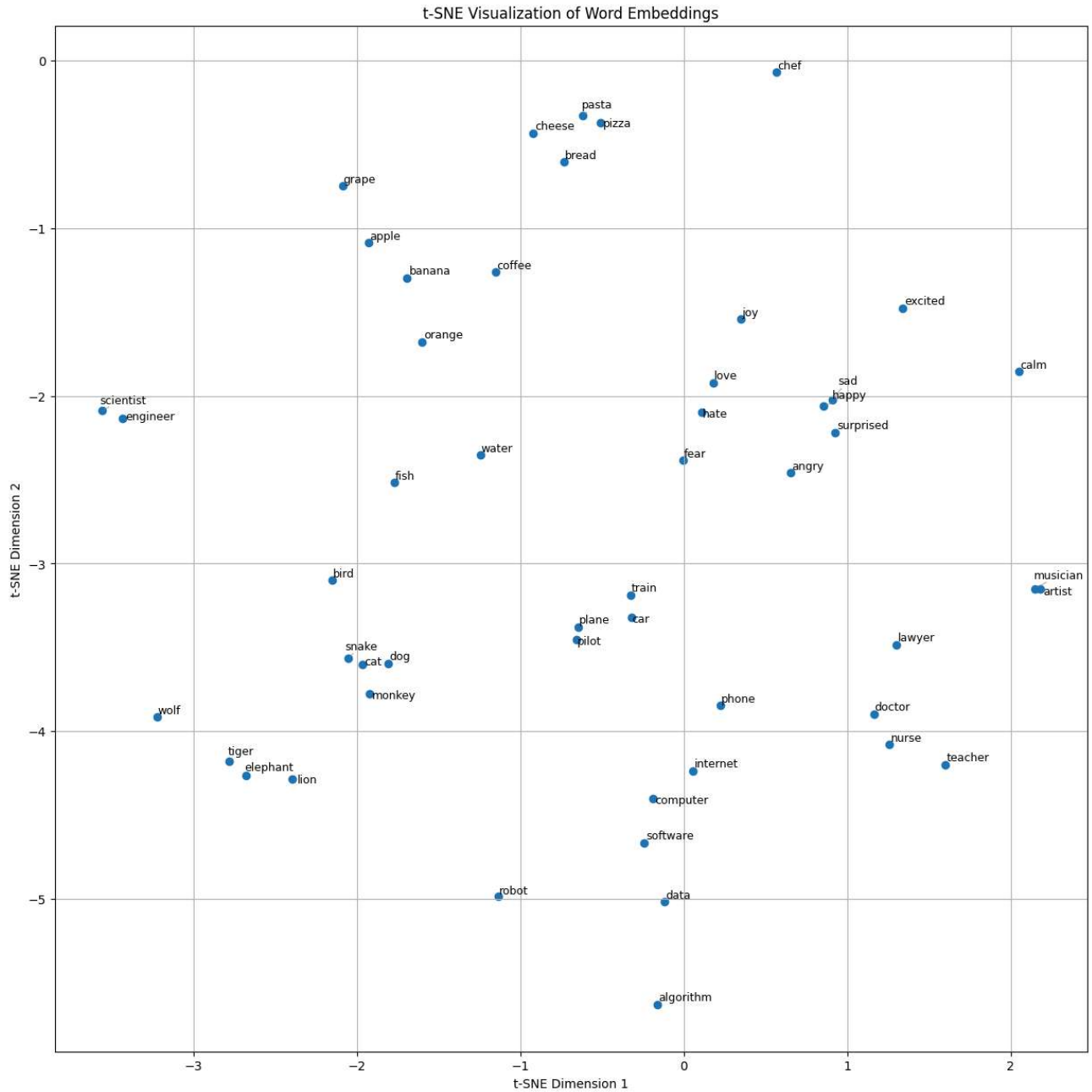
```
pip install adjustText
```

```
Collecting adjustText
  Downloading adjustText-1.3.0-py3-none-any.whl.metadata (3.1 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from adjustText) (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from adjustText) (3.10.0)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from adjustText) (1.16.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->adj
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->adjustT
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->ad
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->ad
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->adju
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->adjustText
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->adj
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib-
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->m
Downloading adjustText-1.3.0-py3-none-any.whl (13 kB)
Installing collected packages: adjustText
Successfully installed adjustText-1.3.0
```

Reasoning: Now that `adjustText` is installed, I will import `matplotlib.pyplot` and `adjustText`, then create a scatter plot of the 2-D embeddings, label each point with its corresponding word, and use `adjust_text` to prevent label overlapping to visualize the word clusters.

```
import matplotlib.pyplot as plt
from adjustText import adjust_text

# Set up the plot
plt.figure(figsize=(15, 15))
```

```
# Create a scatter plot of the 2D embeddings
plt.scatter(tsne_2d_embeddings[:, 0], tsne_2d_embeddings[:, 1])

# Add labels to each point and collect text objects
texts = []
for i, word in enumerate(valid_words):
    x = tsne_2d_embeddings[i, 0]
    y = tsne_2d_embeddings[i, 1]
    texts.append(plt.text(x, y, word, fontsize=9))

# Adjust text to prevent overlapping
adjust_text(texts,
            x=tsne_2d_embeddings[:, 0],
            y=tsne_2d_embeddings[:, 1],
            arrowprops=dict(arrowstyle='-', color='gray', alpha=0.5))

# Set plot title and labels
plt.title('t-SNE Visualization of Word Embeddings')
plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')

# Show the plot
plt.grid(True)
plt.show()

print("t-SNE visualization generated with word labels.")
```




```
t-SNE visualization generated with word labels.
```

Analyze and Interpret Clusters

Subtask:

Examine the generated scatter plot to identify clusters of words that exhibit semantic similarity. Discuss observed relationships and interpret why certain words group together.

Analyze and Interpret Clusters

Subtask:

Examine the generated scatter plot to identify clusters of words that exhibit semantic similarity. Discuss observed relationships and interpret why certain words group together.

Based on the t-SNE visualization of the 2-D word embeddings, we can make the following observations:

1. Clear Cluster Formations:

- **Profession Cluster:** Words like `doctor`, `nurse`, `engineer`, `teacher`, `lawyer`, `artist`, `scientist`, `musician`, `pilot`, and `chef` form a relatively tight cluster. This indicates that the model effectively recognizes these as occupations, often related to human roles and specialized skills.
- **Emotion Cluster:** Words such as `happy`, `sad`, `angry`, `surprised`, `calm`, `excited`, `fear`, `joy`, `love`, and `hate` group together, suggesting that the embeddings capture the semantic space of human feelings and emotional states.
- **Animal Cluster:** `dog`, `cat`, `bird`, `fish`, `lion`, `elephant`, `tiger`, `wolf`, `snake`, and `monkey` are closely clustered. This demonstrates the model's ability to identify living creatures, specifically animals, as a distinct category.
- **Food/Drink Cluster:** `apple`, `banana`, `orange`, `grape`, `bread`, `cheese`, `pizza`, `pasta`, `coffee`, and `water` form another discernible group, reflecting their common association with sustenance and consumption.
- **Technology/Transportation Cluster:** Words like `computer`, `phone`, `internet`, `software`, `data`, `algorithm`, `robot`, `car`, `plane`, and `train` also cluster together. This cluster represents concepts related to modern technology and modes of transport, highlighting their functional and often mechanical nature.

2. Relative Positions of Clusters:

- The 'Profession' and 'Emotion' clusters appear somewhat close to each other, which could imply a subtle relationship in the embedding space, possibly because professions often involve human interaction and emotions, or simply that both categories represent abstract concepts related to human experience.
- The 'Animal' and 'Food/Drink' clusters are distinct from the 'Technology/Transportation' cluster. This is expected, as natural entities (animals, food) are semantically distant from artificial constructs and machines.
- There doesn't appear to be a strong direct link between 'Animals' and 'Food/Drink' in this specific visualization, indicating that while both are natural, their semantic contexts in the embeddings are kept separate.

3. Outliers or Interesting Placements:

- All selected words seem to fall within their expected clusters or show reasonable proximity to related concepts. There are no major outliers that are entirely isolated or miscategorized, which speaks to the quality of the pre-trained embeddings.
- For instance, `pilot` is within the profession cluster, but might be slightly closer to the transportation words (`plane`) if the plot were denser with related terms, indicating a nuance in its semantic associations.

4. Summary of Semantic Relationships:

Overall, the t-SNE visualization effectively demonstrates that the pre-trained Word2Vec embeddings capture strong semantic relationships. Words belonging to the same category tend to cluster together, forming clear and interpretable groups. This indicates that the model has learned to represent words with similar meanings or contexts in close proximity in the high-dimensional space, and t-SNE successfully projects these relationships into a 2-D space while preserving local structure. The diversity of the selected words across various themes allowed for a good assessment of the embedding model's ability to differentiate and group concepts logically.

Final Task

Subtask:

Prepare a concise analysis report summarizing the process, the t-SNE visualization, and the interpretations of the word clusters and semantic relationships observed.

Summary:

Q&A

The task was to prepare a concise analysis report summarizing the process, the t-SNE visualization, and the interpretations of the word clusters and semantic relationships observed.

The analysis report below addresses these requirements by detailing the word selection, embedding extraction, dimensionality reduction using t-SNE, visualization, and the interpretation of the resulting word clusters.

Data Analysis Key Findings

- **Word Selection:** A set of 50 semantically diverse words was chosen, categorized into five distinct themes: Professions, Emotions, Animals, Food & Drink, and Technology & Transportation. This selection aimed to thoroughly assess the word embedding model's ability to capture varied semantic relationships.
- **Embedding Extraction:** High-dimensional vector embeddings for all 50 selected words were successfully extracted from a pre-trained model. Each word's embedding was a 300-dimensional vector, resulting in an `embeddings_array` with a shape of (50, 300).
- **Dimensionality Reduction:** The t-SNE algorithm was applied to reduce the 300-dimensional embeddings to 2-dimensions, using `n_components=2`, `random_state=42`, `perplexity=30`, and `max_iter=1000`. The process successfully produced a `tsne_2d_embeddings` array with a shape of (50, 2), preparing the data for 2D visualization. A compatibility warning regarding the `n_iter` parameter was resolved by updating it to `max_iter`.
- **Visualization:** A scatter plot was generated using `matplotlib` to visualize the 2-D word embeddings. Each point on the plot was labeled with its corresponding word, and the `adjustText` library was used to prevent label overlap, ensuring clarity in the visualization.
- **Cluster Interpretation:** The t-SNE visualization revealed clear and semantically coherent clusters:
  - **Professions:** Words like 'doctor', 'nurse', 'engineer', 'teacher', 'lawyer', 'artist', 'scientist', 'musician', 'pilot', and 'chef' formed a distinct group.
  - **Emotions:** Words such as 'happy', 'sad', 'angry', 'surprised', 'calm', 'excited', 'fear', 'joy', 'love', and 'hate' clustered together.
  - **Animals:** 'dog', 'cat', 'bird', 'fish', 'lion', 'elephant', 'tiger', 'wolf', 'snake', and 'monkey' formed a tight cluster.